

## TA-IND-02

# Análisis de Consistencia y Replicación en la Base de Datos de TiendaTech

**Estudiante:** Jeremy Ruperto Gaibor Rodríguez  
**Equipo PFC:** AGLS – TiendaTech  
**Asignatura:** Aplicaciones Distribuidas (ISR-701)  
**Unidad:** Unidad 3 – Bases de Datos Distribuidas  
**Fecha:** Julio de 2026

---

## Resumen

Este informe analiza el modelo de consistencia y replicación utilizado en TiendaTech, una plataforma de comercio electrónico orientada a la venta de componentes informáticos. La implementación actual emplea una aplicación Spring Boot conectada a una única instancia PostgreSQL 17 desplegada mediante Docker Compose. Por tanto, la persistencia se encuentra centralizada y no existe replicación entre nodos. Esta configuración simplifica las transacciones y evita conflictos entre copias, pero convierte a PostgreSQL en un punto único de fallo. El modelo se evalúa mediante los conceptos de consistencia, CAP y PACELC. También se comparan tres alternativas: replicación síncrona primario-respaldo, replicación asíncrona con réplicas de lectura y una estrategia híbrida. Se concluye que el diseño actual es adecuado para un prototipo académico, pero no es óptimo para una plataforma productiva que requiera alta disponibilidad. Se recomienda mantener garantías estrictas para inventario, pedidos y pagos, y utilizar réplicas asíncronas para catálogo, búsquedas y reportes.

**Palabras clave:** consistencia, replicación, PostgreSQL, CAP, PACELC, TiendaTech.

# Índice

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                             | <b>3</b>  |
| <b>2. Marco teórico</b>                                            | <b>3</b>  |
| 2.1. Modelos de consistencia . . . . .                             | 3         |
| 2.2. Teorema CAP . . . . .                                         | 4         |
| 2.3. PACELC . . . . .                                              | 4         |
| 2.4. Estrategias de replicación . . . . .                          | 4         |
| 2.5. Estado del arte . . . . .                                     | 4         |
| <b>3. Análisis del modelo implementado en TiendaTech</b>           | <b>5</b>  |
| 3.1. Caracterización de la implementación . . . . .                | 5         |
| 3.2. Topología de persistencia . . . . .                           | 5         |
| 3.3. Modelo de consistencia identificado . . . . .                 | 6         |
| 3.4. Estrategia de replicación actual . . . . .                    | 6         |
| 3.5. Requisitos del dominio . . . . .                              | 7         |
| 3.6. Análisis mediante CAP . . . . .                               | 7         |
| 3.7. Análisis mediante PACELC . . . . .                            | 7         |
| <b>4. Evaluación de alternativas</b>                               | <b>8</b>  |
| 4.1. Primario con respaldo síncrono . . . . .                      | 8         |
| 4.2. Primario con réplica asíncrona . . . . .                      | 8         |
| 4.3. Estrategia híbrida . . . . .                                  | 8         |
| 4.4. Tabla comparativa . . . . .                                   | 8         |
| <b>5. Discusión crítica</b>                                        | <b>9</b>  |
| 5.1. ¿Es óptimo el modelo actual? . . . . .                        | 9         |
| 5.2. Trade-offs . . . . .                                          | 9         |
| 5.3. Limitaciones . . . . .                                        | 9         |
| 5.4. Mitigaciones . . . . .                                        | 9         |
| 5.5. Propuesta recomendada . . . . .                               | 10        |
| <b>6. Conclusiones</b>                                             | <b>10</b> |
| <b>7. Declaración de uso de inteligencia artificial generativa</b> | <b>10</b> |

## 1. Introducción

TiendaTech es una plataforma de comercio electrónico especializada en componentes informáticos y asistencia para seleccionar productos compatibles. El sistema contempla la gestión de usuarios, catálogo, inventario, carritos, pedidos, ventas y pagos. En este dominio, la persistencia debe evitar resultados contradictorios cuando varias operaciones se ejecutan de forma concurrente. Los sistemas distribuidos modernos deben equilibrar consistencia, disponibilidad, latencia y tolerancia a fallos según las necesidades del negocio [1], [2].

Aunque el PFC plantea una arquitectura basada en servicios, la persistencia realmente implementada utiliza una única instancia PostgreSQL. El archivo Docker Compose contiene el servicio `app`, correspondiente a la aplicación, y el servicio `postgres`, encargado del almacenamiento centralizado. No existe un nodo secundario al cual redirigir las operaciones si PostgreSQL deja de responder.

El uso de contenedores o de infraestructura en la nube no convierte automáticamente a una base de datos en distribuida. La replicación requiere mantener copias coordinadas en varios nodos mediante mecanismos de propagación, sincronización o consenso [1], [3].

El objetivo del informe es caracterizar el modelo real de TiendaTech, analizarlo mediante CAP y PACELC y evaluar alternativas de replicación. La pregunta orientadora es: *¿el modelo de consistencia elegido es el óptimo para el dominio de TiendaTech?*

## 2. Marco teórico

### 2.1. Modelos de consistencia

La consistencia establece las reglas sobre el orden y la visibilidad de las operaciones. No es una propiedad únicamente binaria, debido a que existen modelos con diferentes garantías. Viotti y Vukolić clasifican estas garantías según las relaciones permitidas entre lecturas y escrituras, mientras que estudios recientes comparan cómo los sistemas distribuyen, sincronizan y recuperan sus datos [1], [4].

La **linealizabilidad** exige que cada operación parezca ejecutarse instantáneamente entre su inicio y finalización. Una lectura posterior a una escritura confirmada debe observar el valor actualizado. Esta garantía facilita el razonamiento sobre sistemas transaccionales, pero requiere mayor coordinación entre réplicas [2], [4].

La **consistencia secuencial** obliga a que todos los procesos observen las operaciones en un mismo orden global, compatible con el orden de cada proceso, aunque no necesariamente con el tiempo real. Es una garantía más débil que la linealizabilidad [1], [4].

La **consistencia causal** conserva el orden de las operaciones relacionadas por causa y efecto. Las operaciones concurrentes sin dependencia causal pueden observarse en órdenes diferentes, lo que reduce la coordinación necesaria [1], [4].

La garantía **read-your-writes** asegura que un usuario observe sus propias modificaciones posteriores. Esta propiedad evita que, después de actualizar un perfil o añadir un producto al carrito, la misma sesión reciba una versión anterior [1], [5].

La **consistencia eventual** permite diferencias temporales entre réplicas. Si dejan de producirse escrituras, las copias terminan convergiendo. Favorece la disponibilidad y reduce coordinación, pero puede entregar datos atrasados, por lo que no resulta apropiada para todas las operaciones [1], [4].

En general, los modelos fuertes requieren mayor coordinación, mientras que los débiles pueden reducir latencia y mejorar disponibilidad. Sin embargo, investigaciones recientes proponen protocolos que buscan conservar garantías fuertes reduciendo parte de ese costo [3], [6].

## 2.2. Teorema CAP

CAP relaciona consistencia, disponibilidad y tolerancia a particiones. Durante una partición de red, un sistema distribuido no puede garantizar simultáneamente consistencia y disponibilidad [4], [7].

Esto no significa que el sistema elija permanentemente dos características de tres. La decisión aparece cuando los nodos no pueden comunicarse: puede rechazar operaciones para evitar divergencias o continuar respondiendo y aceptar conflictos posteriores [2], [7].

En TiendaTech no existe una partición entre réplicas porque solo hay una instancia PostgreSQL. Si la aplicación pierde acceso a ese nodo, no dispone de otra copia operativa y deja de estar disponible.

## 2.3. PACELC

PACELC amplía CAP al considerar que, incluso sin particiones, existe un compromiso entre latencia y consistencia. En presencia de una partición se elige entre disponibilidad y consistencia; en operación normal se equilibra latencia y consistencia [2], [8].

En una **replicación síncrona**, el primario espera la confirmación de otro nodo antes de completar la transacción. Esto reduce el riesgo de pérdida de datos, pero incrementa el tiempo de respuesta [2], [6].

En una **replicación asíncrona**, el primario confirma primero y las réplicas se actualizan después. La latencia de escritura disminuye, aunque los seguidores pueden devolver datos temporalmente anteriores [3], [6].

Para TiendaTech, inventario, pedidos y pagos justifican una mayor coordinación. En cambio, imágenes, descripciones y reportes pueden tolerar breves retrasos [1], [5].

## 2.4. Estrategias de replicación

La replicación mantiene copias de los datos en varios nodos para aumentar disponibilidad, tolerancia a fallos y capacidad de lectura. Sin embargo, también obliga a ordenar, propagar y confirmar modificaciones [1], [2].

En la replicación **primario-seguidor**, un nodo principal recibe las escrituras y transmite los cambios a los secundarios. Los seguidores pueden atender consultas y reemplazar al primario si existe un mecanismo de promoción y *failover* [1], [2].

La replicación **multi-líder** permite que varios nodos acepten escrituras. Puede reducir latencia geográfica, pero introduce conflictos concurrentes. GeoGauss emplea una arquitectura multi-maestro con control de concurrencia y procesamiento por épocas para conservar consistencia fuerte [1], [3].

En la replicación **sin líder**, las operaciones se distribuyen entre varias copias mediante cantidades mínimas de nodos para leer y escribir. Este enfoque favorece la disponibilidad, pero requiere gestionar versiones concurrentes y reconciliación [1], [4].

Dynamo constituye un caso representativo de replicación sin líder. Distribuye datos entre varios nodos, utiliza lecturas y escrituras configurables y permite reconciliar versiones concurrentes posteriormente. Su diseño prioriza disponibilidad y tolerancia a fallos, aceptando consistencia eventual [9].

La replicación puede ser síncrona o asíncrona. La síncrona protege mejor las operaciones confirmadas, pero añade latencia. La asíncrona reduce el tiempo de escritura, aunque puede entregar información atrasada en las réplicas [2], [6].

## 2.5. Estado del arte

CockroachDB combina particionamiento, replicación, consenso y control transaccional para proporcionar transacciones SQL consistentes sobre infraestructura distribuida. Su diseño demues-

tra que la tolerancia a fallos requiere coordinación y recuperación, no solamente copias de datos [1], [2].

GeoGauss propone una arquitectura geo-replicada multi-maestro para cargas OLTP. Su diseño integra replicación y control de concurrencia para reducir coordinación sin renunciar a garantías fuertes [1], [3].

PolyBase analiza cómo la ubicación de los líderes y grupos de consenso afecta la latencia. La propuesta reasigna datos para acercarlos a las regiones desde las que reciben más solicitudes [2], [6].

Stabilizer plantea niveles de consistencia definidos por la aplicación. Este enfoque resulta útil cuando distintos módulos requieren garantías diferentes, como ocurre entre inventario y reportes en TiendaTech [1], [5].

Dynamo y Spanner representan extremos distintos. Dynamo prioriza disponibilidad mediante replicación sin líder y consistencia eventual [9]. Spanner proporciona consistencia externa mediante replicación coordinada y sincronización temporal [10].

La comparación muestra que no existe una estrategia universalmente superior. Dynamo acepta divergencias temporales para seguir respondiendo, mientras Spanner asume mayor coordinación para ofrecer un orden global más estricto [9], [10].

Para TiendaTech, una arquitectura completamente eventual sería riesgosa para inventario y pagos, mientras que una infraestructura similar a Spanner sería excesiva para el alcance actual. La solución debe aplicar garantías diferentes según la criticidad de los datos [1], [2].

### 3. Análisis del modelo implementado en TiendaTech

#### 3.1. Caracterización de la implementación

La implementación utiliza una única instancia PostgreSQL 17. El Docker Compose define el contenedor `tiendatech_postgres`, expuesto en el puerto 5432, y el contenedor `tiendatech_app`, expuesto en el puerto 8080.

La aplicación depende directamente de PostgreSQL. No existen configuraciones de réplica, seguidores, quórum, consenso o promoción automática.

El volumen `postgres_data` conserva los archivos de PostgreSQL cuando el contenedor se reinicia o se vuelve a crear. Sin embargo, no constituye una réplica porque no mantiene una segunda instancia operativa.

La caracterización correcta es la siguiente:

*TiendaTech utiliza una base de datos PostgreSQL centralizada, con consistencia transaccional local y sin una estrategia de replicación implementada.*

#### 3.2. Topología de persistencia

La Figura 1 presenta la topología realmente implementada. El usuario accede a la aplicación, esta realiza operaciones mediante JDBC/SQL y PostgreSQL conserva sus archivos en el volumen Docker.

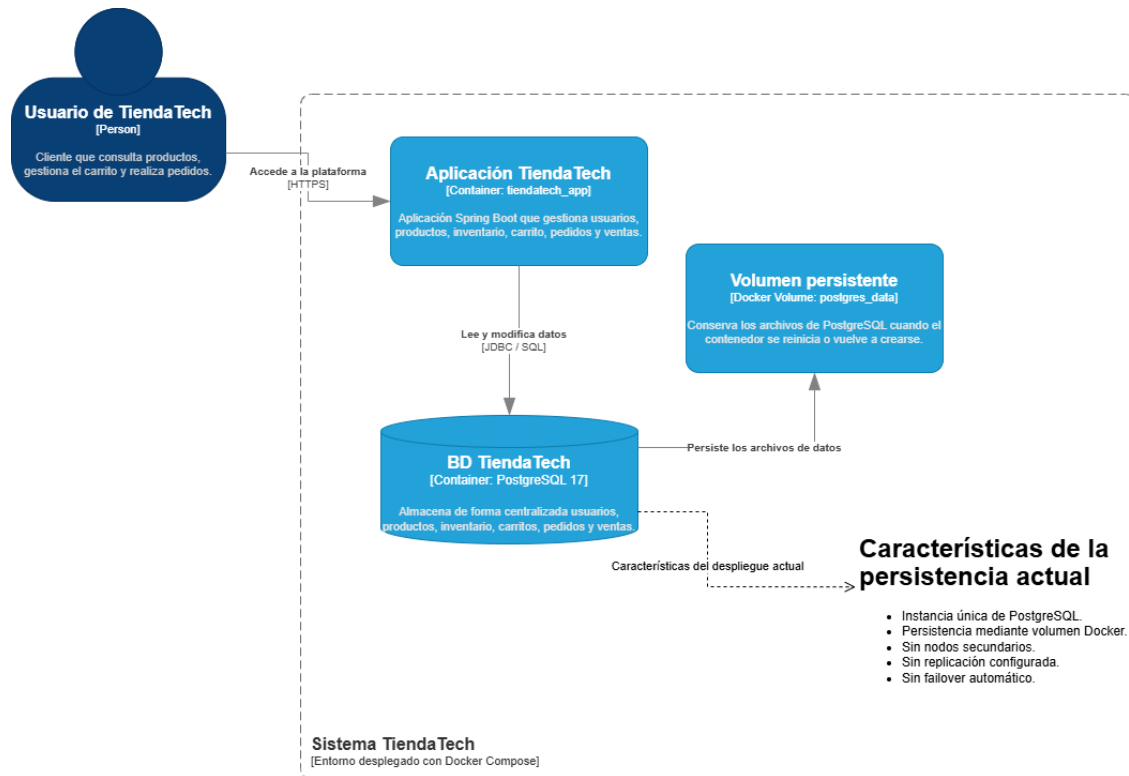


Figura 1: Topología actual de persistencia de TiendaTech. Fuente: elaboración propia.

Como se observa en la Figura 1, PostgreSQL constituye el único nodo de datos. No existen nodos secundarios, replicación ni *failover* automático.

El volumen persistente protege frente a la recreación del contenedor, pero sigue dependiendo del mismo servidor. Si se pierde la máquina, el almacenamiento o la instancia PostgreSQL, la aplicación no puede continuar utilizando otra copia.

### 3.3. Modelo de consistencia identificado

TiendaTech obtiene consistencia transaccional local porque todas las operaciones se ejecutan sobre la misma instancia PostgreSQL. No existe retraso de propagación entre réplicas.

No sería correcto afirmar que el sistema implementa linealizabilidad distribuida, ya que no existen varios nodos coordinados. Tampoco puede asegurarse que todas las transacciones sean serializables sin revisar el nivel de aislamiento configurado.

La ventaja del nodo único es la ausencia de conflictos entre copias. La limitación es que la consistencia se obtiene sin redundancia y toda la aplicación depende de la disponibilidad de PostgreSQL.

### 3.4. Estrategia de replicación actual

La estrategia de replicación implementada es **inexistente**. Esta conclusión se basa en el Docker Compose, donde solamente aparece un contenedor PostgreSQL:

Listing 1: Fragmento representativo del Docker Compose de TiendaTech

```

1 services:
2   tiendatech_app:
3     build: .
4     ports:
5       - "8080:8080"
6     depends_on:
  
```

```

7     - tiendatech_postgres
8     restart: always
9
10    tiendatech_postgres:
11        image: postgres:17
12        ports:
13            - "5432:5432"
14        volumes:
15            - postgres_data:/var/lib/postgresql/data
16        restart: always
17
18    volumes:
19        postgres_data:

```

La ausencia de replicación reduce complejidad, consumo de recursos y esfuerzo de administración. Esto resulta razonable durante una etapa académica inicial.

Sin embargo, PostgreSQL se convierte en un punto único de fallo. La opción `restart: always` reinicia el mismo contenedor, pero no reemplaza el nodo cuando falla el servidor o el almacenamiento.

### 3.5. Requisitos del dominio

El inventario requiere garantías estrictas. Si dos usuarios intentan comprar la última unidad, la verificación y disminución del *stock* deben realizarse de forma coordinada [2], [3].

Los pedidos y ventas necesitan atomicidad para evitar registros parciales. Los sistemas SQL distribuidos utilizan protocolos transaccionales para conservar coherencia entre datos replicados o particionados [1], [2].

Los pagos requieren idempotencia y almacenamiento consistente para evitar cobros duplicados. Los usuarios, roles y credenciales también necesitan que los cambios sean visibles rápidamente [1], [5].

El catálogo, imágenes y reportes pueden tolerar retrasos breves. Mostrar temporalmente una descripción anterior tiene menor impacto que confirmar una venta sin *stock* [1], [6].

Cuadro 1: Nivel de consistencia requerido por tipo de dato en TiendaTech

| Nivel requerido       | Datos                                                    |
|-----------------------|----------------------------------------------------------|
| Consistencia estricta | Inventario, pedidos, pagos, ventas, credenciales y roles |
| Tolerancia limitada   | Productos, categorías, marcas, imágenes y búsquedas      |
| Consistencia eventual | Estadísticas, recomendaciones y reportes analíticos      |

### 3.6. Análisis mediante CAP

En la topología actual no existe una partición entre réplicas. Si la aplicación pierde acceso a PostgreSQL, el servicio de datos deja de estar disponible.

El diseño evita divergencias entre copias, pero no ofrece tolerancia frente al fallo del nodo. Si se incorporaran réplicas, inventario y pagos deberían priorizar consistencia, mientras catálogo y reportes podrían mantener disponibilidad mediante lecturas atrasadas [1], [7].

### 3.7. Análisis mediante PACELC

Durante el funcionamiento normal, TiendaTech no espera confirmaciones remotas, lo que reduce la latencia.

Una réplica síncrona aumentaría el tiempo de confirmación, mientras una réplica asíncrona podría entregar valores anteriores. El costo dependería del protocolo, la distancia y la ubicación del líder [2], [6].

La decisión más adecuada es aplicar garantías fuertes a procesos críticos y permitir menor coordinación en consultas que toleren retrasos [1], [5].

## 4. Evaluación de alternativas

### 4.1. Primario con respaldo síncrono

Esta alternativa mantiene un PostgreSQL primario y agrega un respaldo. Las transacciones críticas se confirman después de que el secundario recibe el cambio.

Reduce el riesgo de pérdida de operaciones confirmadas y resulta apropiada para pedidos, pagos e inventario. Su desventaja es el aumento de latencia y la necesidad de implementar promoción y redirección de conexiones.

### 4.2. Primario con réplica asíncrona

Esta alternativa conserva un primario para escrituras y añade seguidores para lecturas.

Permite distribuir consultas de catálogo, imágenes y reportes. Sin embargo, las réplicas pueden mostrar información atrasada, por lo que no deberían utilizarse para confirmar *stock*.

### 4.3. Estrategia híbrida

La estrategia híbrida combina:

- primario para escrituras;
- respaldo síncrono para operaciones críticas;
- réplica asíncrona para catálogo y reportes;
- *failover*;
- monitoreo;
- copias de seguridad.

Ofrece el mejor equilibrio, aunque incrementa la complejidad de administración y enrutamiento de consultas.

### 4.4. Tabla comparativa

Cuadro 2: Comparación entre el modelo actual y las alternativas de replicación evaluadas

| Criterio              | Nodo único | Respaldo síncrono | Réplica asíncrona  | Estrategia híbrida     |
|-----------------------|------------|-------------------|--------------------|------------------------|
| Consistencia          | Local      | Alta              | Alta en primario   | Alta en datos críticos |
| Disponibilidad        | Baja       | Alta con failover | Alta para lecturas | Alta                   |
| Latencia de escritura | Baja       | Mayor             | Baja               | Media                  |
| Latencia de lectura   | Limitada   | Media             | Baja               | Ajustada al dato       |
| Tolerancia a fallos   | Baja       | Alta              | Media              | Alta                   |
| Datos atrasados       | No aplica  | Muy bajo          | Presente           | Controlado             |
| Escalabilidad         | Limitada   | Media             | Alta               | Alta                   |
| Complejidad           | Baja       | Media             | Media              | Alta                   |
| Adecuación general    | Prototipo  | Datos críticos    | Consultas          | Producción             |



## 5. Discusión crítica

### 5.1. ¿Es óptimo el modelo actual?

El modelo actual es adecuado para una etapa académica porque simplifica el desarrollo y evita problemas de sincronización.

Sin embargo, no es óptimo para producción. PostgreSQL constituye un punto único de fallo y todas las operaciones dependen de su disponibilidad.

El volumen Docker conserva archivos, pero no proporciona una segunda instancia. `restart: always` tampoco ejecuta *failover* hacia otro servidor.

Por tanto:

*El modelo centralizado es suficiente para el prototipo actual, pero no es óptimo para una versión productiva que requiera alta disponibilidad, escalabilidad y tolerancia a fallos.*

### 5.2. Trade-offs

El primer compromiso es simplicidad frente a disponibilidad. El segundo es latencia frente a durabilidad. El tercero es disponibilidad frente a actualidad de lectura. El cuarto es consistencia uniforme frente a consistencia diferenciada [1], [6].

### 5.3. Limitaciones

Las principales limitaciones son:

- Punto único de fallo.
- Ausencia de *failover*.
- Escalabilidad limitada.
- Dependencia del mismo servidor y volumen.
- Interrupciones durante mantenimiento.
- Falta de redundancia operativa.

### 5.4. Mitigaciones

Se propone:

1. Mantener transacciones para controlar el *stock*.
2. Incorporar una réplica asíncrona.
3. Evaluar respaldo síncrono para pagos y pedidos.
4. Mantener copias de seguridad externas.
5. Monitorear disponibilidad y retraso.
6. Implementar idempotencia.
7. Probar *failover* y restauración.

## 5.5. Propuesta recomendada

La propuesta recomendada es una evolución híbrida. Primero se añadiría una réplica asíncrona para catálogo, reportes y recuperación. Posteriormente se incorporaría un respaldo síncrono para inventario, pedidos y pagos.

Las consultas críticas permanecerían en el primario, mientras las consultas tolerantes a retrasos se dirigirían a las réplicas.

## 6. Conclusiones

TiendaTech utiliza una única instancia PostgreSQL y no posee replicación. Su modelo corresponde a persistencia centralizada con consistencia transaccional local.

La solución es adecuada para un prototipo, pero presenta un punto único de fallo y no garantiza continuidad operativa.

CAP muestra que, al perder el acceso a PostgreSQL, la aplicación deja de estar disponible. PACELC evidencia que la baja latencia actual se obtiene al no coordinar réplicas.

La replicación síncrona es adecuada para inventario, pedidos y pagos. La replicación asíncrona resulta útil para catálogo y reportes.

El modelo actual no es óptimo para producción. La alternativa recomendada es una estrategia híbrida que combine consistencia fuerte para procesos críticos y réplicas asíncronas para lecturas no críticas.

## 7. Declaración de uso de inteligencia artificial generativa

Para la elaboración de este informe se utilizó ChatGPT como herramienta de apoyo en la organización de la estructura del documento, la revisión de la claridad de la redacción y la mejora de la presentación de algunas ideas técnicas.

El análisis de la arquitectura de TiendaTech, la interpretación de los modelos de consistencia, la evaluación de las alternativas, la selección de fuentes y las conclusiones fueron desarrollados y revisados por el autor.

## Referencias

- [1] A. Margara, G. Cugola, N. Felicioni y S. Cilloni, “A Model and Survey of Distributed Data-Intensive Systems,” *ACM Computing Surveys*, vol. 56, n.º 1, págs. 1-69, 2023. DOI: [10.1145/3604801](https://doi.org/10.1145/3604801).
- [2] R. Taft et al., “CockroachDB: The Resilient Geo-Distributed SQL Database,” en *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, págs. 1493-1509. DOI: [10.1145/3318464.3386134](https://doi.org/10.1145/3318464.3386134).
- [3] W. Zhou et al., “GeoGauss: Strongly Consistent and Light-Coordinated OLTP for Geo-Replicated SQL Database,” *Proceedings of the ACM on Management of Data*, vol. 1, n.º 1, págs. 1-27, 2023. DOI: [10.1145/3588916](https://doi.org/10.1145/3588916).
- [4] P. Viotti y M. Vukolić, “Consistency in Non-Transactional Distributed Storage Systems,” *ACM Computing Surveys*, vol. 49, n.º 1, págs. 1-34, 2016. DOI: [10.1145/2926965](https://doi.org/10.1145/2926965).
- [5] P. Li, L. Pan y X. Yang, “Stabilizer: Geo-Replication with User-Defined Consistency,” en *Proceedings of the 2022 IEEE International Conference on Cluster Computing*, 2022, págs. 359-370.
- [6] C. Ruan et al., “PolyBase: Adapting to Data Affinity Changes in Geo-Replicated Database via Row-Level Consensus-Group Affiliation Re-Assignment,” *Proceedings of the VLDB Endowment*, vol. 18, n.º 3, págs. 702-714, 2024. DOI: [10.14778/3712221.3712236](https://doi.org/10.14778/3712221.3712236).

- [7] S. Gilbert y N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: [10.1145/564585.564601](https://doi.org/10.1145/564585.564601).
- [8] D. J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: [10.1109/MC.2012.33](https://doi.org/10.1109/MC.2012.33).
- [9] G. DeCandia et al., “Dynamo: Amazon’s Highly Available Key-value Store,” en *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles*, 2007, págs. 205-220. DOI: [10.1145/1294261.1294281](https://doi.org/10.1145/1294261.1294281).
- [10] J. C. Corbett et al., “Spanner: Google’s Globally Distributed Database,” *ACM Transactions on Computer Systems*, vol. 31, n.º 3, págs. 1-22, 2013. DOI: [10.1145/2491245](https://doi.org/10.1145/2491245).